



### PRACTICAL - 5

**AIM :** Implementation of a knapsack problem using dynamic programming.

#### **ALGORITHM :**

Knapsack

Input :-  $n$  items with weights  $w[i]$  and values  $v[i]$ , and knapsack capacity  $w$ .

Output :- Maximum value that can be obtained.

Step 1 :- Start

Step 2 :- Read the no. of items  $n$  and knapsack capacity  $w$ .

Step 3 :- Read the weight  $w[i]$  and value  $v[i]$  of each item

Step 4 :- Initialize the first row and first column of the DP table to 0

Step 5 :- loop for  $i = 1$  to  $n$

Step 6 :- loop for  $j = 1$  to  $w$ .

Step 7 :- if  $w[i] \leq j$ , calculate :  
 $take = v[i] + dp[i-1][j-w[i]]$

Step 8 :- calculate : not take  $= dp[i-1][j]$

Step 9 :- set:  
 $dp[i][j] = \max(take, not\ take)$

Step 10 :- if  $w[i] > j$ , set

Step 11 :-  $dp[n][w]$

Step 12 :- Trace the  $dp[n][w]$

Step 13 :- Stop.

**PROGRAM :**

```
#include <iostream>
using namespace std;

class Knapsack
{
private:
    int weight[10];
    int value[10];
    int n;
    int capacity;

    int dp[10][100];

public:

    // Take input
    void getInput()
    {
        cout << "Enter number of items: ";
        cin >> n;

        cout << "Enter knapsack capacity: ";
        cin >> capacity;

        for (int i = 1; i <= n; i++)
        {
            cout << "Enter weight and value of item " << i << ": ";
            cin >> weight[i] >> value[i];
        }
    }

    // Calculate maximum value
    void solve()
    {
        // Initialize first row and first column with 0
        for (int i = 0; i <= n; i++)

        {
            dp[i][0] = 0;
        }

        for (int j = 0; j <= capacity; j++)
        {
```

```
dp[0][j] = 0;
}

// Fill the DP table
for (int i = 1; i <= n; i++)
{
    for (int j = 1; j <= capacity; j++)
    {
        if (weight[i] <= j)
        {
            // Option 1: Take the item
            int take = value[i] + dp[i - 1][j - weight[i]];

            // Option 2: Don't take the item
            int notTake = dp[i - 1][j];

            // Choose the better option
            dp[i][j] = max(take, notTake);
        }
        else
        {
            // Item is too heavy
            dp[i][j] = dp[i - 1][j];
        }
    }
}

// Print DP table
void printTable()
{
    cout << "\nDP Table:\n";

    for (int j = 0; j <= capacity; j++)
    {
        cout << j << "\t";
    }
    cout << endl;

    for (int i = 0; i <= n; i++)
    {
        for (int j = 0; j <= capacity; j++)
        {
```



```
        cout << dp[i][j] << "\t";
    }

    cout << endl;
}

// Find which items were selected
void findItems()
{
    cout << "\nSelected Items:\n";

    int j = capacity;

    for (int i = n; i >= 1; i--)
    {
        // If value changed, item was selected
        if (dp[i][j] != dp[i - 1][j])
        {
            cout << "Item " << i
                << " -> Weight = " << weight[i]
                << ", Value = " << value[i]
                << " -> TAKEN\n";

            j = j - weight[i];
        }
        else
        {
            cout << "Item " << i
                << " -> NOT TAKEN\n";
        }
    }
}

// Display final answer
void showResult()
{
    cout << "\nMaximum Value = " << dp[n][capacity] << endl;
}

};

int main()
{
```

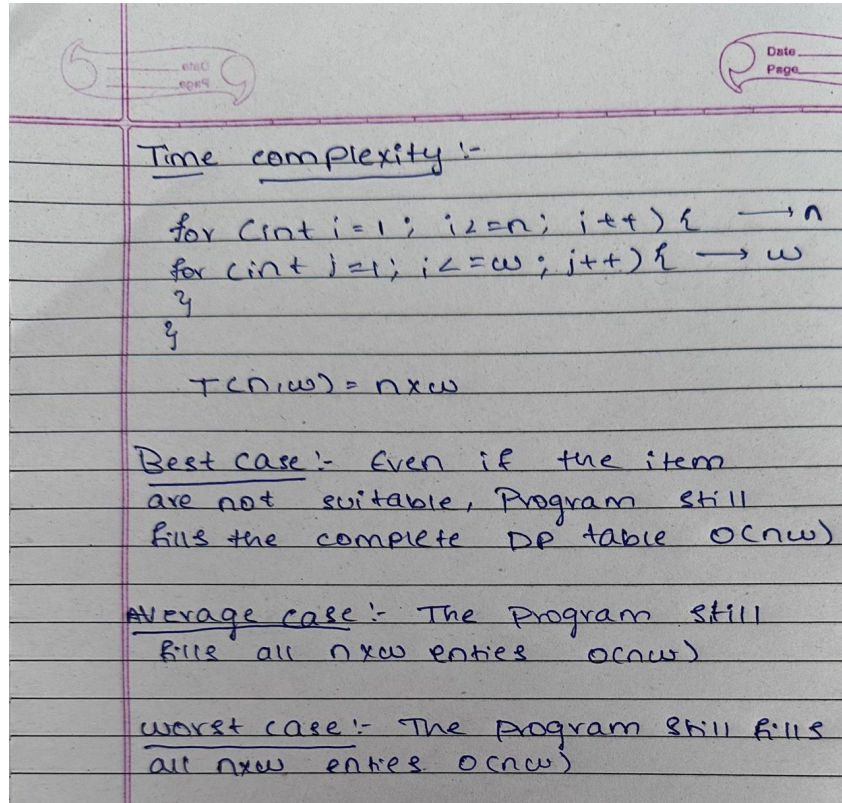
```
Knapsack k;  
  
k.getInput();  
  
k.solve();  
  
k.printTable();  
  
k.showResult();  
  
k.findItems();  
  
return 0;  
}
```

## OUTPUT :

```
SYNC STORAGE/UNIVERSITY STUFF/DAA_LAB  
● > ./Knapsack  
Enter number of items: 4  
Enter knapsack capacity: 8  
Enter weight and value of item 1: 2 1  
Enter weight and value of item 2: 3 2  
Enter weight and value of item 3: 4 5  
Enter weight and value of item 4: 5 6  
  
DP Table:  
0   1   2   3   4   5   6   7   8  
0   0   0   0   0   0   0   0   0  
0   0   1   1   1   1   1   1   1  
0   0   1   2   2   3   3   3   3  
0   0   1   2   5   5   6   7   7  
0   0   1   2   5   6   6   7   8  
  
Maximum Value = 8  
  
Selected Items:  
Item 4 -> Weight = 5, Value = 6 -> TAKEN  
Item 3 -> NOT TAKEN  
Item 2 -> Weight = 3, Value = 2 -> TAKEN  
Item 1 -> NOT TAKEN
```



## TIME COMPLEXITY



**CONCLUSION :** I successfully implemented the 0/1 Knapsack problem using the given weights and profits. Through this practical, I understood how to select items optimally within a given capacity to obtain the maximum possible profit. I also learned how dynamic programming can be used to solve the knapsack problem efficiently by breaking it into smaller subproblems.